



Report

Robot Controller

Dinh-Son Vu

Table of Contents

1. Pre-requisites	3
2. Tasks	3
3. Explanations	4
4. Packages, nodes, and topics	5
5. Future works	5

1. Pre-requisites

- Ubuntu 24.04 Noble can run on your computer (VM, UTM, or dual boot).
- ROS2 Jazzy is installed.
- This section is in simulation only. The hardware is not required.

2. Tasks

Previously, the robot could be visualized in Rviz and gazebo, but could not move. This package rmitbot_controller introduces the controller manager. It loads the following concepts:

- teleopkeyboard: the keyboard can be used to control the robot in the Cartesian space.
- Controller manager: the core of the package. It supervises the input/output of the robot.
- Joint_state_broadcaster_spawner: part of the controller manager. It starts the node to read the state of the robot (wheel position and velocity)
- Controller_spawner: part of the controller manager. It starts the node to translate the Cartesian velocity command to wheel velocity command.

Tasks	Code
Install ros2_control and ros2_controller	sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
Install gz_ros2_control	sudo apt-get install ros-jazzy-gz-ros2-control
Install teleopkeyboard	sudo apt install ros-jazzy-teleop-twist-keyboard
Install external terminal	sudo apt install xterm
Clone the repo from github	git clone https://github.com/ROS2-rmitbot/lesson2_ws.git
Navigate to the workspace	cd ~/lesson2_ws
Remove the folders build install log	rm -rf build install log
Build the packages	colcon build
Source the workspace	source install/setup.bash
Open VSCode	code .
Launch rviz, gazebo, controller, teleopkeyboard	ros2 launch rmitbot_description display.launch.py ros2 launch rmitbot_description gazebo.launch.py ros2 launch rmitbot_controller controller.launch.py ros2 launch rmitbot_controller teleopkeyboard.launch.py
Alternatively, launch the bringup file	ros2 launch rmitbot_bringup rmitbot.launch.py
Check the publisher/subscriber of a topic	ros2 topic info -v <yourtopic> e.g. ros2 topic info -v /rmitbot_controller/cmd_vel
Use the command line from the previous lesson	ros2 node list ros2 topic list ros2 run rqt_graph rqt_graph

```

teleop_twist_keyboard

Moving around:
u i o
j k l
m , .

For Holonomic mode (strafing), hold down the shift key:

U I O
J K L
M < >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently: speed 0.5 turn 1.0

```

Figure 1: The teleopkeyboard.

3. Explanations

The package `ros2_control` is used to move the robot in the simulated environment with the keyboard. Later, we will see how to go from simulation to real hardware, but for the moment, you should get comfortable with the concept of nodes and topics. The parameter `use_sim_time` has been set to `true` for all nodes since gazebo is used (simulation environment). When real hardware is used, then `use_sim_time` will be set to `false`. The packages have been modified as follows:

- `rmitbot_description`:
 - o `urdf/rmitbot_ros2_control.xacro`: this file describes which part of the robot is controlled and which sensors send data to the computer. It contains: `command_interface`, `state_interface`, and gazebo plugins.
 - o `launch/display.launch.py`: the joint state publisher gui is disabled. We no longer need it as `ros2_control` is used to publish the `joint_state` (position and velocity of the joint).
- `rmitbot_controller`
 - o `config/rmitbot_controller.yaml`: configuration of the `ros2_controller` package
 - o `joint_state_broadcaster`: this node is broadcasting `/joint_state`, which is essential for `robot_state_publisher`, which broadcasts the `tf` of the robot.
 - o `rmitbot_controller`: it loads the node `diff_drive_base_controller`. This node publishes the topic `/rmitbot_controller/odom`, which is the main way of getting the pose of the robot from encoders. It subscribes to a twist `/rmitbot_controller/cmd_vel` and performs the inverse kinematics, to send the command to the motors of the robot.
 - o `teleopkeyboard`: Use of keyboard to control the robot in gazebo.

4. Packages, nodes, and topics

Table 1: packages of the lesson 2

Packages	Nodes	Description
rmitbot_bringup		Launch the other packages and nodes
rmitbot_description	display.launch.py: rviz, rsp gazebo.launch.py: gazebo	Launch the urdf and static tf
rmitbot_controller	Controller manager: jsb spawner, controller spawner	Launch input/output of the robot

Table 2: nodes and topics of the lesson 2

Package	Topic	Description
ROS2 standard	/parameter_events /rosout	n/a n/a
rviz	/clicked_point /goal_pose /initialpose	Click point in rviz Target pose of the robot (for nav2) Initial pose of the robot (for slam_toolbox)
gazebo	/clock	Sim time for node synchronization
robot_state_publisher	/robot_description /tf /tf_static	Description of the robot in xml format Dynamic tf (odom --> base_footprint) Static tf (base_footprint --> rest of the robot)
controller_manager	/controller_manager/activity /controller_manager/introspection_data/full /controller_manager/introspection_data/names /controller_manager/introspection_data/values	For debugging only
ros2_controllers	/joint_states /rmitbot_controller/cmd_vel /rmitbot_controller/odom /rmitbot_controller/cmd_vel_out /joint_state_broadcaster/transition_event /dynamic_joint_states /rmitbot_controller/transition_event	Important: pos, vel, effort of the joint Important: Cartesian vel command Important: odometry (pose of the robot)

5. Future works

- The simulated robot can be controlled with the keyboard, which is fine for debugging. The use of a joystick would be appreciated.
- The simulated robot can move, but the speed is unknown. One can use the tool plotjuggler to visualize and record important states, e.g., wheel velocities